

THE CLASS STORE

One lesson, and the whole week is built to land it.

You cannot check software unless you already know the answer.

Your class works out *by hand* what a full sell-out should earn. Then the app works it out. On day 5 the cash tin, the paper ledger and the app give three different numbers, and the class has to say which one is lying and why.

That is what the five days are for. They decide what to sell and what it costs, you build the storefront live in front of them, and they run it for real.

5

days, and every one feeds day 5

3

numbers on the last day that
should match

0

accounts, logins or
subscriptions

The students never touch the chatbot – and still own what it was told to do.

Chatbots are 13 and over, and no child's name goes into a prompt. So students do the research, the pricing, the feature list, the testing and the decisions, and **you do the prompting, on the board, where they can see their own spec becoming software**. Specification and judgment are the actual thinking, and **you cannot check software against your own number unless you know what it was asked for.**

Inside

Day-by-day teacher pages · **seven student pages worth printing** · the exact prompts written out in full · what to do when the AI breaks it in front of thirty children · **the working store app, included and ready**, with a setup screen so you never edit code · and the page on what this store deliberately refuses to record.

The week at a glance

Day	The class does	You do
1 What sells	Survey the room. Rank what people want against what they will actually pay.	Model a bad survey question, then a good one.
2 Price and stock	Set prices and opening stock across three tiers. Work out a full sell-out by hand.	Push back on pricing. "Because it is cool" is not a reason.
3 Build it	Read the spec aloud, watch it become a store, call out what is wrong.	Prompt on the board. Let it fail and get fixed where they can see.
4 Open	Run a real selling session. Serve customers. Write down everything that goes wrong.	Say nothing. Write down every workaround you watch them invent.
5 Count up	Tin against ledger against app. Explain the gap.	Make their one change live. Ship it.

What each day is actually for.

Every other class store ends at "we had a store." This one is five steps toward one sentence: **you cannot check software unless you already know the answer.**

Day 1 gets a real number instead of a guess. **Day 2** works out the sell-out *by hand* — the number day 5 checks against. **Day 3** puts their spec into software, so they know what it was told to do. **Day 4** produces the paper ledger. **Day 5** puts tin, ledger and app side by side and makes them explain the gap.

If you say one thing all week, say "we are going to check the computer."

How long it really takes

Days 1, 2 and 5 are a lesson each. **Day 3 is 25 minutes** and fits at the end of another lesson. **Day 4 is as long as your store session is** — twenty minutes is plenty. The days need not be consecutive, and it works better spread over two weeks.

What you need

One computer you can project. Any AI chat tool you already use. A tin and whatever you use as class money. **Your download is one zip** — unzip it once and the store file opens by double-click, with nothing installed.

Two rules, and one of them is not negotiable

1 • The grown-up holds the keyboard

Every mainstream chatbot requires users to be 13 or over, and school AI guidance is consistent that a child's name, work or identity should not be typed into one. So **no student sits at the chatbot** at any point in this unit.

This is not a workaround. Students who spend the week deciding *what the software must do* and then testing whether it does it are doing the harder and more transferable half of the work.

2 • The class decides, the AI builds

When a student asks for something and the AI does it differently, **the student is right and the AI is wrong** — say that out loud on day 3. The specification is theirs. Your job at the keyboard is to be a fast typist with opinions about what is possible, not the person who decides what the store should do.

And one thing to settle before day 1

Class money, not real money.

This unit is written for whatever token currency your room already uses. **If real money is involved, that is school funds** — handling, counting and banking it goes through your office, under their rules, and nothing in this pack is advice about that. The arithmetic lesson is exactly the same either way, so use class money if you have the choice.

The three-tier shelf, and why the store is built around it

Classroom-store practice converges on three price bands, and the app is built to expect all three:

Tier 1 — free privileges. Sit anywhere for a day, choose the music, hat for the day. They cost you nothing and they are usually the most wanted things on the shelf.

Tier 2 — small things, bought most weeks. Stickers, pencils, erasers. This is what keeps the store busy.

Tier 3 — one big thing to save for. Lunch outside with a friend, the prize box. **A store with no tier 3 gets emptied in the first session and stops motivating anybody by week two.** This is the single most common way a class store dies, and it is worth telling the class so on day 2.

What will actually sell

One lesson. Ends with a ranked list of what this class will really spend on, built from evidence rather than from the loudest three voices.

Why this day exists: day 5 compares three numbers. Today is where the class learns that **a number you guessed and a number you measured are not the same thing**, which is the habit the whole week runs on.

1 Ask the wrong question first (4 minutes)

Put this on the board: **"Would you like a class store?"** Every hand goes up. Then ask what that told you. Answer: nothing. A question everybody says yes to has no information in it. **This is the whole lesson in one move** and it lands better as a demonstration than as a rule.

2 Then the right one (6 minutes)

Build a better question together. The test is that a reasonable person could answer it differently from the person next to them. **"Which one of these three would you spend 20 on?"** passes. "Do you like stickers?" does not.

3 Run the survey (12 minutes)

Student page on the next sheet. Each student asks five people, inside the class or in another class. **Forced choice, not a rating.** Ratings make everything look popular; choosing forces a trade-off, which is what a price is.

4 Pool it and rank it (10 minutes)

Tally on the board. Now the argument starts, and this is the good part: **the most popular thing is not automatically the thing to stock.** Ask what happens if the most popular item is the one that costs you the most to provide.

The sentence to end on.

"We are not stocking what we like. We are stocking what this room will actually pay for." Write it up. It is the difference between a business and a wish list, and they will quote it back at each other on day 2.

If it derails

Somebody will propose something impossible — a phone, a day off. **Do not rule it out from the front.** Ask what it would cost you to provide, and let them work out that it is not on. **Letting the numbers kill an idea teaches them the numbers matter.**

What will people really spend on?

Ask **five** people. Read them the choice exactly as it is written. Do not explain it, and do not tell them which one you picked.

My three things to choose between

A

B

C

My question, word for word: "If you had 20 to spend, which one of these three would you buy?"

What they said

Person	Chose	What they said about why
1		
2		
3		
4		
5		

The one thing that surprised me

Watch out for this. If all five picked the same thing, your three choices were not close enough. Two bad options and one good one is not a survey, it is a trick.

Price it and stock it

One lesson. Ends with a priced, stocked shelf across three tiers **and a number worked out by hand** that the software will later be checked against.

1 Teach the three tiers (6 minutes)

Page 3 has them. Give the class the rule and let them sort their day 1 list into the bands. **Insist on at least one tier 3.** If nobody wants to save for anything, the store is a vending machine and it stops working in a fortnight.

2 Price it, and defend it (12 minutes)

Every price needs a reason that is not "because it is cool." The two reasons that count: **what it costs us to provide**, and **what people said they would spend on day 1**. Make groups defend three prices to the room.

3 Stock it (5 minutes)

Opening stock for each item. Tier 1 privileges can be effectively unlimited. Tier 3 should be genuinely scarce — **one or two** — or it is not worth saving for.

4 The hand arithmetic (12 minutes) — do not skip this

The sell-out sheet is on page 8. **Every group works out what the shelf earns if every single item sells.** Pens and paper. Then collect the numbers and put them on the board where they will still be on day 5.

Why the hand arithmetic is the point of day 2.

On day 5 the software will tell them a number. If nobody has ever worked one out themselves, that number is just whatever the computer said. **Doing it by hand first is what makes checking possible** — and checking is the skill this unit is actually teaching.

The disagreement worth having

Two groups will get different sell-out totals. **Do not resolve it.** Make them find the difference between their two methods. It is nearly always somebody forgetting the tier 1 privileges, which is a useful thing to discover before the software does it for them.

Our shelf, in three tiers

Every price needs a reason. **"Because it is cool" is not a reason.** The two that count are what it costs us to provide, and what people said on day 1.

Tier 1 – free privileges (costs us nothing to give)

What it is	Price	Stock	Why that price

Tier 2 – small things, bought most weeks

What it is	Price	Stock	Why that price

Tier 3 – the one big thing to save up for

What it is	Price	Stock	Why that price

Check before you hand this in. Is there at least one tier 3? Is the tier 3 price high enough that somebody has to save for more than one week to reach it? If not, put it up.

What does a full sell-out earn?

Work this out **on paper, before the computer does it**. On day 5 we are going to check the software against your number, and we cannot do that unless you have one.

Every item on our shelf, if every single one sells

Item	Price	Stock	Price × stock

Our sell-out total

Who checked it

Two questions to answer underneath

Is that a realistic amount for this class to spend in one term? If it is far more than anybody could earn, the prices are too high or the stock is too big.

Which single item is the biggest share of that total? If one thing is most of it, the store depends on one thing selling.

Build it live, in twenty-five minutes

Why this day exists: you cannot check software against your own number unless you know **what the software was told to do**. Today they say it, out loud, and watch it get built.

The class watches their own spec become a working store. **You are at the keyboard. They are the ones who say whether it is right.**

Do this before the lesson, not during it

Open `class-store.html` from this download, click **Set up the store**, and check it opens. That is all. **The store already works** — day 3 is about making it theirs, not about building software from nothing in front of a class, which is how a live build goes wrong.

1 Put their shelf in, on the board (10 minutes)

Open the setup screen and type their day 2 shelf into it — name, price, stock, tier. **No code, no file editing**. Let them read the prices out to you and correct you when you mistype. The correcting is the point: it is their shelf.

2 Or let the AI do it, and show them the catch (5 minutes)

The setup screen has a prompt that turns their list into a price list in about thirty seconds. Use it, then **read the result against the day 2 sheet in front of them** and find where it changed a price or invented a tier. That is the lesson: fast is not the same as right, and somebody still has to check.

3 Change one real thing by asking (8 minutes)

Page 10 has the prompts. Pick **one**, paste the whole file into your AI tool, ask for the change, save the result as a new file and open it. One change. Play with it. If it broke, page 11 is the recovery.

4 Save it (2 minutes)

Save the store as a file. This is the step people skip and regret — it hands back one file with their shelf inside it, which is what you open on day 4.

Say this out loud when the AI does something they did not ask for.

"It did not do what we said. We are right and it is wrong." Then fix it in front of them. **A class that watches you correct a computer learns something a class that watches you accept its output does not.**

Written out in full, to copy

Paste the whole `class-store.html` file into whichever AI tool you use, then one of these. **Always save the result as a new file** and keep the old one.

1 · MAKE IT LOOK LIKE OUR ROOM

Here is a working HTML class store. Change the colors to **[our school colors, named]** and make the product names larger so they can be read from the back of a classroom on a projector. Do not change how anything works. Give me the whole file back.

2 · A SALE LIMIT, WHICH EVERY CLASS ASKS FOR BY DAY 4

In this file, stop one customer buying more than **three** of the same item in a single sale. Show a clear message saying why rather than just disabling the button. Change nothing else, and tell me which part you changed so I can change the number myself next time.

3 · A TWO-FOR-ONE DEAL ON ONE ITEM

Add an offer to this store: when a customer buys two of **[item name]** in the same sale, the second one is half price. It must show up in the cart total and in the ledger, so the numbers still add up at the end of the day. Give me the whole file back.

4 · A RESTOCK BUTTON

Add a way, on the set-up screen only, to add more stock to an item without changing its price or clearing the ledger. Follow exactly how the existing set-up fields are written.

Three rules that will save you an afternoon.

One change per prompt, and open it before you ask for the next. Always say *give me the whole file back*, or you get a fragment and nowhere to put it. And **always save under a new name** — then a result you dislike costs you nothing at all.

What not to ask for, and the reason

Do not ask it to add student names, per-student balances, or a record of who bought what. Page 19 explains why in full, and it is the one request this unit turns down on purpose.

When it breaks in front of thirty children

It will, once. Handled well it is the best ninety seconds of the week, because they watch an adult be wrong in public and fix it calmly.

What you see	What to do, out loud
The new file opens blank	"Something in the change broke it." Close it, open the copy you saved, carry on. Thirty seconds. Then paste the broken file back and say: <i>this stopped working after your change, fix only that.</i>
It changed something you did not ask for	Name it: "We asked for one thing and it did two." Ask again with the extra thing forbidden explicitly. This is the most common failure and the easiest to narrate.
It gives you a fragment instead of a file	You forgot <i>give me the whole file back</i> . Ask again with that sentence. Do not try to paste a fragment in.
It argues that your idea is bad	Good moment. "It is not the customer. We are." Restate the instruction without justifying it.
It is taking too long and the room is going	Stop. "We will finish this at lunch and you will see it tomorrow." A live build has a time box and it is better to end it than to lose the room.
The prices come out wrong	Back to the day 2 sheets. The class's paper number is the authority, not the screen. Say that.

The one thing not to do.

Do not quietly fix it later and show them a working version next lesson. **The repair is the content.** A class that only ever sees finished software learns that software arrives finished, which is the opposite of what this unit is for.

Open the store

Why this day exists: the paper ledger your students keep today is **the second of the three numbers** day 5 compares. Without it there is nothing to check the app against but the tin.

A real selling session. Twenty minutes is plenty. **Your job today is to say nothing and write things down.**

1 Set the room up first (5 minutes, before they arrive)

The store file open on one machine. The tin. The printed price list — the Ledger tab prints one. **And the paper ledger on page 14**, because the paper record is what makes day 5 possible.

2 Give out three jobs

The till works the app. **The paper ledger** writes down every sale independently. **The queue** keeps order and watches the clock. Rotate halfway. Everybody else is a customer, and the customers are the ones being studied.

3 Then stop talking (the rest of it)

Watch for **workarounds** — the moments somebody does something the software did not intend. Writing a total on their hand. Asking a friend to hold their place. Serving two people at once. **Every workaround is a design finding** and it is worth more than anything they will tell you when you ask.

The rule, and it is hard to keep.

If you have to explain how it works, that is a finding, not a customer problem. Write down what you had to explain instead of explaining it better. The observation sheet opposite is for exactly this.

The thing that will go wrong, and let it

Somebody will try to buy the last one of something at the same moment as somebody else, or the queue will find an item that is sold out and argue about it. **Do not pre-empt this.** The app refuses the sale and says so, which is correct, and the argument about it is day 5's material.

One laptop, thirty customers

Serve in two waves by table rather than a single queue, or let the queue job hand out numbered slips. Decide it before you start; discovering it live is the one piece of chaos that is not instructive.

Watch the shop floor

You are not serving. You are watching. **Write down what people do, not what they say.** A customer who says it was fine and then does something odd has told you it was not fine.

Workarounds I saw – somebody doing something the store did not expect

Things somebody had to be told out loud

Every one of these is something the store should have made obvious by itself.

Where the queue stopped moving

Something a customer wanted that we do not sell

Do not write "it went well." That is not a finding and nobody can do anything with it. Write down one exact thing that one exact person did.

The paper ledger

Write down **every** sale as it happens, separately from the computer. On day 5 we compare the two. If they match, that tells us something. If they do not, that tells us more.

#	What was sold	How many	Total paid
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

Paper total

Sales we nearly missed

Write down the sale nobody rang up too. If somebody was served while the till was busy, that is the most useful line on this page — it is usually the whole gap on day 5.

Count up, and find out why

The lesson the rest of the week exists for. Ends with a gap explained and **one** change made.

1 Count the tin, before you open the laptop (5 minutes)

Two students count it separately and must agree. **Do not look at the app yet.** If the screen is showing a number while they count, the number they get will be the number on the screen.

2 Three numbers on the board (5 minutes)

What the **tin** holds. What the **paper ledger** says. What the **app** says, from the Count up tab. Write all three up before anybody explains anything.

3 Find the gap (15 minutes) – the actual lesson

They will assume the computer is right. **Say clearly that it is not automatically right**, and that the three numbers are three witnesses, not one truth. The student page opposite walks the hunt. The usual causes, in the order they turn up:

A sale served while the till was busy and never rung up · change given wrong · a shelf price that did not match the price in the app · somebody buying two and it being recorded as one · a refund nobody recorded.

4 Decide one change, and make it (10 minutes)

One. Not a list. They argue, they pick, you make it live at the keyboard while they watch, using a prompt from page 10 or the setup screen. Then save the file.

The sentence to close the unit on.

"The software was not wrong. It recorded exactly what we told it. **The gap was us.**" Every adult who has ever reconciled anything knows this and almost no eleven-year-old has been told it.

If the numbers match exactly

It happens, and it is not a failure of the lesson. Ask what it proves: that the arithmetic agrees. Ask what it does not prove: that every sale was rung up. **Two errors can cancel.** Then ask how they would know, which is a better question than the one they would have answered.

Three numbers, and the hunt

The computer is not automatically right — it recorded exactly what somebody told it. **Find where the three numbers stopped agreeing.**

The tin

The paper ledger

The app

The biggest gap is between

and it is **how much?**

Work down this list. Tick what you checked.

Did this happen?

What we found

Somebody was served while the till was busy

Change was given wrong

A shelf price did not match the app price

Two were bought and one was recorded

Something was given back and nobody wrote it down

So the gap was caused by

More in the tin than the app says is still a gap, and still worth explaining. It usually means a sale was taken but never rung up.

Change one thing

Not a list. **One.** You have to argue for it and you have to say what it costs.

The one change we are asking for

The evidence from day 4 that says we need it

Something you or somebody else actually watched happen. Not "it would be better if."

What it costs us

More time at the till? A slower queue? Something else we now cannot do? Every change costs something. If you cannot name the cost, you have not thought it through yet.

What we decided NOT to change, and why

This is the grown-up part of the page.

How to tell a good change from a bad one. A good change stops something that actually happened. A bad one prevents something nobody has ever done. **The store does not need a rule for every possible thing** — only for what went wrong in this room, on day 4.

Everything goes in from one screen

Open `class-store.html` by double-clicking it. **You never edit code and you never open it in a text editor.** Press **Set up the store** and it is all there.

The shelf

A row per product: what it is, the price, the opening stock, and which of the three tiers it sits in. Add and remove rows with the buttons. The store refuses to remove a product that has already sold something, because that would break the ledger — set its stock to zero instead, which is what you actually mean.

The AI shortcut, and the catch worth teaching

The same screen gives you a prompt to copy. Paste it into your AI tool with the class's list underneath and it hands back a full price list in about thirty seconds, which you paste straight back in. **Read it against the day 2 sheets before you sell from it.** It will occasionally change a price or invent a tier, and finding that in front of the class is the best two minutes of day 3.

Selling, and the last one on the shelf

Tap items to build a cart, watch the total, finish the sale. **Stock goes down as things sell and the last one stops selling**, with a message rather than a silent failure. Undo the last sale is on the Ledger tab for the moment somebody rings up the wrong thing.

Count up, and the print

Count up is day 5: type what you counted and it shows the difference and what that difference usually means. **Print the price list** is on the Ledger tab and prints the shelf, tier by tier, for the wall.

Save the store as a file, and do it on day 3.

It hands you back one file with your shelf and your ledger inside it. Keeping it in the browser also works until somebody clears the browsing data, which happens. **A file on your own drive survives that, opens on any computer, and is still there next September.**

What it refuses to do, and why

There is no box anywhere in this store for a student's name, and there is no per-student balance. That is deliberate, and the reason is worth teaching rather than hiding.

A record of who bought what is a record about a child

It is the first feature every class asks for and it is the one to turn down. A list of which named child bought what, when, and how much they had left is a **spending and behavior record** — and unlike a paper ledger in your drawer, a file gets emailed, synced and copied onto other machines without anybody deciding to do that. So the store tracks **stock and totals**, and nothing else.

Where things live, in three lines

Build freely

Products, prices, stock, tiers, the store's name, the price list, the class totals. None of it is about a child, and the class can share any of it.

On your own machine

The saved store file and the ledger of what sold. Keep them where you keep your own work. Nothing is transmitted anywhere, because there is nowhere to send it and nobody to send it to.

Stop, and ask

Any of it attached to a named child. Balances per student. Anything synced to a shared drive, emailed around, or pasted into a chatbot. **And real money** — if what is in the tin is legal tender rather than class money, handling and banking it goes through your office under their rules, and nothing in this pack is advice about that.

If you want balances, keep them on paper.

Not a compromise — a better answer. A paper ledger in your drawer belongs to you, does not get copied, and disappears when you shred it. That is a stronger guarantee than any setting in any app.

What actually goes wrong, and the sentence that fixes it

What happens	What to do
The store sells eleven of ten pencils	It will not — the last one stops selling and says so. If a class insists it happened, somebody sold from the paper ledger while the app was on another tab. That is a day 5 finding, not a bug.
The shelf is empty when you open it	You opened the original file rather than the one you saved. Find the file you downloaded on day 3. This is why saving has its own step.
Everything disappeared between lessons	It was kept in the browser and the browsing data was cleared, or you are on a different computer. Open your saved file. Then save it again to your drive rather than the desktop.
The pasted product list did not load	Every line needs four parts separated by the character. The message names the line numbers it skipped and why. An AI that numbered the lines is the usual cause: delete the numbers.
It will not let you remove a product	That product has sold something and removing it would break the ledger. Set its stock to zero.
It will not replace the whole shelf	Same reason. Add to the shelf instead, or clear the ledger first if you are genuinely starting a new term.
The file will not open at all	Right-click, Open with, pick a browser. A school laptop sometimes tries to open an HTML file in a text editor.
An AI change broke it	Delete that file and open the copy you kept. This is the whole reason page 10 says to save under a new name.

The failure that is not a failure.

On day 4 something will go wrong with the queue, the change or the recording. **Let it.** A selling session where nothing goes wrong gives day 5 nothing to work on, and day 5 is the lesson.

What you can actually grade here

No rubric with nine columns. **Four things this unit produces evidence of**, each tied to a page you already have in your hand.

What you are looking at	The evidence	Where
Evidence over opinion	Did they change a price because of what the survey said, or defend it because they liked it? The "why that price" column is the whole assessment.	Day 2 shelf page
Arithmetic they can defend	A sell-out total worked by hand, and whether they can say which item is the biggest share of it.	Day 2 sell-out page
Observing rather than asking	Did they write down something a person <i>did</i> , or something a person <i>said</i> ? This is the hardest skill on the list and the easiest to see.	Day 4 observation page
Reasoning about a discrepancy	Can they explain a gap without blaming the computer, and name what they did not change as well as what they did?	Day 5 pages

The single best question to ask a student on day 5

"How would you know if you were wrong?" A student who can answer that about their own price, their own count, or their own change has got the thing this unit is for. It also happens to be the question that cannot be answered by copying anybody.

What not to grade

Do not grade the takings. A class that sold more did not learn more, and the moment the money is the mark, the pricing stops being honest and the day 5 count stops being safe to report accurately. **The gap on day 5 has to be a safe thing to find.**

Running it again, and what changes

Next term, with the same class

Open your saved file, go to **Set up the store** and press **Clear the ledger and start a new term**. The shelf and the prices stay; the sales go. Then hand day 2 back to them: **which prices were wrong?** They have evidence now, which they did not have the first time, and the second pricing lesson is better than the first.

Next year, with a new class

Start from the original file again, not from last year's saved one. **The shelf has to be theirs or none of this works** — a class selling last year's products at last year's prices is running somebody else's shop, and day 2 becomes a copying exercise.

If it goes well and they want more

The honest next steps are not more features. They are: **a second selling session with the one change made**, to see whether it worked — which is the real discipline and the thing almost no classroom project ever does. Then a different question entirely: what would we stock if we had to earn the money to buy the stock?

The two requests to turn down

Per-student balances — page 19, and the answer is a paper ledger. **Letting a student do the prompting** — page 3, and the answer is that they can dictate every word of the prompt to you, which is the same learning without the account.

The measure of whether this worked.

Not the takings, and not whether the store still runs in March. It is whether, the next time somebody in that room is handed a number by a computer, one of them asks where it came from.

Things that already exist, and what is different here

Digital classroom economies are not new and pretending otherwise is the fastest way to lose a review. Here is the comparison, in both directions.

What is already out there

There are free digital classroom-economy platforms where students earn class money and spend it in a digital store, and paid behavior platforms that do the same alongside points and reports. **They are more finished than this, they handle balances for you, and if what you want is a running class economy they may well be the better tool.** Say so.

What is different about this one

Your class designed it. That is the whole unit. Students cannot learn specification from software somebody else specified — the surveying, pricing, testing and deciding is the lesson, and a finished platform does all of it for them.

Nobody has an account. No child signs up for anything, no child's name exists in the file, and nothing is transmitted.

It is a file you own. It works with the wifi down, it cannot be blocked or switched off, there is nothing to renew, and every rule in it can be changed the morning you decide it is wrong.

What this pack is not

Not a course in programming. Not a full classroom-economy system — there is no earning side, no payroll and no banking, on purpose. Not advice about handling real money. And not a claim that the store app is better software than a product with a company behind it. **It is the store your class specified, which is a different thing entirely.**

On building on somebody else's idea

Shops, tills and ledgers belong to nobody and you are welcome to build on them. The words, the design and the code in these files are ours. That distinction is worth knowing the moment a class starts changing software with AI: **how a thing works is free to borrow, the particular way somebody said and drew it is not.**

License, credit, and one favor

What you may do

Run this unit with your own classes for as long as you teach. Print the student pages as many times as you need. Change the store however you like and save as many versions as you want. Put the file on the classroom computers or a flash drive. Your one purchase covers you and your classes.

What needs another license

Sharing it with other teachers, putting it on a shared drive or a server, or handing it round a department or a building. Additional licenses are half price on the listing, and a school can buy them on a single purchase order. Please do not post the files publicly or sell anything made from them.

What was tested before this shipped

The store app was driven start to finish in a browser: a sale to the last unit with the sold-out refusal, a pasted product list with a bad line reported rather than swallowed, the ledger, a deliberate discrepancy in Count up, saving to a file and reopening it with the shelf and ledger intact. **The refusals were tested as carefully as the successes,** because those are the ones that show up in front of a class.

If this earned its keep, would you leave a review?

TPT gives you credit toward your next purchase for it, and it is the only thing that decides whether another teacher ever sees this. If something did not work in your room, tell us that instead and we will fix it.

Built and audited by two certified teachers

Written by classroom teachers, checked by a second teacher before it shipped, and built on a rule we keep everywhere: the grown-up holds the keyboard, and no child's name goes into a prompt.